

ADITYA COLLEGE OF ENGINEERING AND TECHNOLOGY

Department of Computer Science & Engineering

SHOPEZ

E-Commerce Web Application

Full Stack Project Documentation — MERN Stack

A Project Report Submitted in Partial Fulfillment of the Requirements for the Internship Program

Submitted By

1. Bonda Kavya Sri — 23MH1A05L9
2. Dwarampudi Bhagyasri — 23MH1A05M1
3. Kaki Guna Sai Teja — 23MH1A05M3
4. Kottela Sandeep — 23MH1A05M6
5. Kunisetty Hadassa — 23MH1A05M8

Academic Year: 2026

Submission Date: July 2026

Table of Contents

Submitted By	1
Table of Contents	2
1. Introduction	4
2. Project Overview	4
Purpose	4
Features	4
3. Architecture	5
Frontend	5
Backend	5
Database	6
4. Setup Instructions	6
Prerequisites	6
Installation	6
5. Folder Structure	7
Client Folder Layout	7
Server Folder Layout	7
6. Running the Application	7
Terminal 1 — Backend Server	7
Terminal 2 — Client Application	7
7. API Documentation	8
8. Authentication	9
9. User Interface & Screenshots	9
9.1 Registration Page	9
9.2 Login Page	10
9.3 Home Page / Product Catalog	10
9.4 Product Search & Filter Results	11
9.5 Product Detail Page	11
9.6 Shopping Cart Page	12
9.7 Checkout Page	12
9.8 My Orders Page	13
9.9 Order Detail Page	13
9.10 Admin Dashboard	14

9.11 Admin – Manage Products	14
9.12 Admin – Manage Orders	15
10. Testing	16
11. Known Issues	16
12. Future Enhancements	16

1. Introduction

Project Title: ShopEZ – MERN E-Commerce Platform

Team: This project was developed collaboratively by a 5-member team, covering full-stack development (frontend and backend), REST API design, JWT-based authentication, MongoDB schema design, and admin dashboard implementation. Individual contributions are listed on the cover page.

2. Project Overview

Purpose

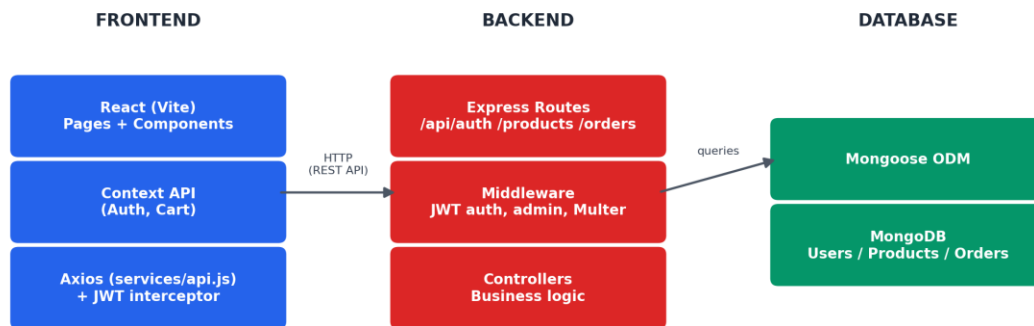
ShopEZ is a full-stack e-commerce web application built to provide a complete online shopping experience — from product discovery through checkout and post-purchase order tracking — while also giving store administrators the tools to manage the product catalog and fulfill orders. The project was built to demonstrate practical, production-grade full-stack engineering patterns using the MERN stack: secure authentication, role-based access control, server-side validation of client-submitted data, and a clean separation between the presentation, API, and data layers.

Features

- **Authentication:** JWT-based authentication. On login/register, the server issues a signed access token (7-day expiry) which the client stores and attaches to subsequent requests via an Authorization header.
- **Product Catalog:** Searchable and filterable catalog. Search matches product name and description (via a MongoDB text index); category filtering is fully dynamic — the category list is derived live from existing product data rather than hardcoded.
- **Shopping Cart:** Client-side cart state (React Context, persisted to localStorage), so it survives page refreshes without requiring a database write until checkout.
- **Checkout:** Cash on Delivery (COD) order flow with server-side re-validation of price and stock for every item — the server never trusts prices submitted by the client. No online payment gateway (credit/debit card, UPI, etc.) is integrated in the current version.

- **Order Tracking:** Customers can view their order history and track each order's status (Pending → Processing → Shipped → Delivered, or Cancelled).
- **Customer Reviews & Ratings:** Logged-in users can leave a 1-5 star rating and written comment on any product (one review per user per product); the product's average rating and review count update automatically.
- **Admin Dashboard:** Full product CRUD, including direct image upload from the admin's computer (stored on the server and served statically); full order visibility with the ability to update order status.

3. Architecture



Uploaded product images stored in server/uploads and served statically at /uploads/<file>

Fig 3.1 – Technical Architecture

Frontend

The client is a Single Page Application built with React 18 and Vite. Routing is handled with React Router. Application-wide state is limited to two concerns — the logged-in user and the shopping cart — both managed with React's built-in Context API rather than a dedicated state-management library, since a heavier solution (e.g. Redux) was not warranted by the app's actual state complexity. All HTTP communication goes through a single shared Axios instance with a request interceptor that automatically attaches the JWT to every call.

Backend

The server is a REST API built with Express.js on Node.js, written in plain JavaScript (CommonJS modules). Routes, controllers, models, and middleware are separated into their own folders by responsibility rather than by business domain. Incoming requests pass through CORS handling,

JSON body parsing, and — for protected routes — JWT verification and role-checking middleware, before reaching the relevant controller function.

Database

MongoDB, accessed through Mongoose. A compound text index on the Product collection (name, description, category) supports catalog search. Reviews are embedded directly inside each Product document, and each Order embeds a frozen snapshot of its line items and shipping address at the time of purchase, rather than referencing live, mutable data. There is no separate Cart collection or Mongoose transaction usage — the cart lives only in the browser until checkout, at which point a single `Order.create()` call converts it into a permanent record.

4. Setup Instructions

Prerequisites

- Node.js (v18 or higher) and npm
- A MongoDB instance (local installation, or a MongoDB Atlas cloud database)
- Git

Installation

1. Clone or download the project codebase locally.
2. Create a `.env` file in the server folder based on the provided `.env.example` (set `MONGO_URI`, `JWT_SECRET`, `PORT`), and a `.env` file in client based on its `.env.example` (`VITE_API_URL`).
3. Install dependencies:

```
# In server root
cd server
npm install

# In client root
cd ../client
npm install
```

4. (Optional) Populate the database with sample data:

```
cd server
```

```
npm run seed
```

5. Folder Structure

Client Folder Layout

```
client/
├── src/
│   ├── components/ # Navbar, ProductCard, StarRating,
│   │               # AdminRoute, PrivateRoute
│   ├── pages/      # Home, ProductDetail, Cart, Checkout,
│   │               # Login, Register, Orders, OrderDetail,
│   │               # AdminDashboard, AdminProducts, AdminOrders
│   ├── context/    # AuthContext.jsx, CartContext.jsx
│   ├── services/   # api.js (Axios instance), imageUrl.js
│   ├── utils/      # currency.js
│   ├── App.jsx     # Route declarations
│   └── main.jsx    # Entry point, provider composition
├── index.html
├── vite.config.js
└── package.json
```

Server Folder Layout

```
server/
├── models/          # User.js, Product.js, Order.js (Mongoose schemas)
├── controllers/     # authController.js, productController.js, orderController.js
├── routes/          # authRoutes.js, productRoutes.js, orderRoutes.js,
uploadRoutes.js
├── middleware/      # auth.js (JWT + admin check), upload.js (Multer),
errorHandler.js
├── config/          # db.js (MongoDB connection)
├── seed/            # seedProducts.js
├── utils/           # generateToken.js
├── uploads/         # uploaded product image files (served statically)
├── .env.example
└── server.js        # Application entry point
```

6. Running the Application

Terminal 1 — Backend Server

```
cd server
npm run dev
```

The API listens on <http://localhost:5000>

Terminal 2 — Client Application

```
cd client
npm run dev
```

The client is served on `http://localhost:3000`

Health check: visit `http://localhost:5000/api/health` — a successful response returns `{"status":"ok","message":"ShopEZ API is running"}`.

7. API Documentation

The key endpoints exposed by the server API are listed below. All endpoints are prefixed with the base URL `http://localhost:5000`.

Method	Endpoint	Description	Auth Required
POST	<code>/api/auth/register</code>	Register a new customer account	No
POST	<code>/api/auth/login</code>	Authenticate and receive a JWT	No
GET	<code>/api/auth/profile</code>	Get the logged-in user's profile	Yes
GET	<code>/api/products</code>	List products (supports keyword, category, page, limit)	No
GET	<code>/api/products/:id</code>	Retrieve a single product's details, including reviews	No
GET	<code>/api/products/categories/all</code>	List all distinct product categories	No
POST	<code>/api/products</code>	Create a new product	Yes (Admin)
PUT	<code>/api/products/:id</code>	Update an existing product	Yes (Admin)
DELETE	<code>/api/products/:id</code>	Delete a product	Yes (Admin)
POST	<code>/api/products/:id/reviews</code>	Submit a star rating and review for a product	Yes
POST	<code>/api/upload</code>	Upload a product image file	Yes (Admin)
POST	<code>/api/orders</code>	Submit a new order (checkout)	Yes
GET	<code>/api/orders/my</code>	Fetch the logged-in user's own order history	Yes
GET	<code>/api/orders</code>	Fetch every order on the platform	Yes (Admin)

Method	Endpoint	Description	Auth Required
GET	/api/orders/:id	Fetch a single order (owner or admin only)	Yes
PUT	/api/orders/:id/status	Update an order's fulfillment status	Yes (Admin)
GET	/api/health	Basic API health check	No

8. Authentication

ShopEZ implements JWT-based authentication with a single access token (no refresh token rotation). On successful login or registration, the server signs a JSON Web Token containing the user's ID and role, with a 7-day expiry, using a server-side secret (JWT_SECRET). The token is returned in the response body and stored client-side in localStorage; an Axios request interceptor automatically attaches it as a Bearer token on every subsequent API call. Protected routes verify this token via the protect middleware, and admin-only routes additionally check the user's role via the admin middleware. When the token expires, the user is required to log in again — there is currently no silent background refresh mechanism.

9. User Interface & Screenshots

The images below are dummy placeholders. Replace each one with an actual screenshot of your running application, in the same order, before submitting. Capture the full browser window (address bar visible) so it is clear the app is running locally on the ports listed in Section 6.

9.1 Registration Page

Signup form showing name, email and password fields.

ShopEZ

Home Cart 1 Login Register

Create Account

Name

Email

Password

Register

Already have an account? [Login](#)

9.2 Login Page

Login form and any validation/error message state.

ShopEZ

Home Cart 1 Login Register

Login

Email

Password

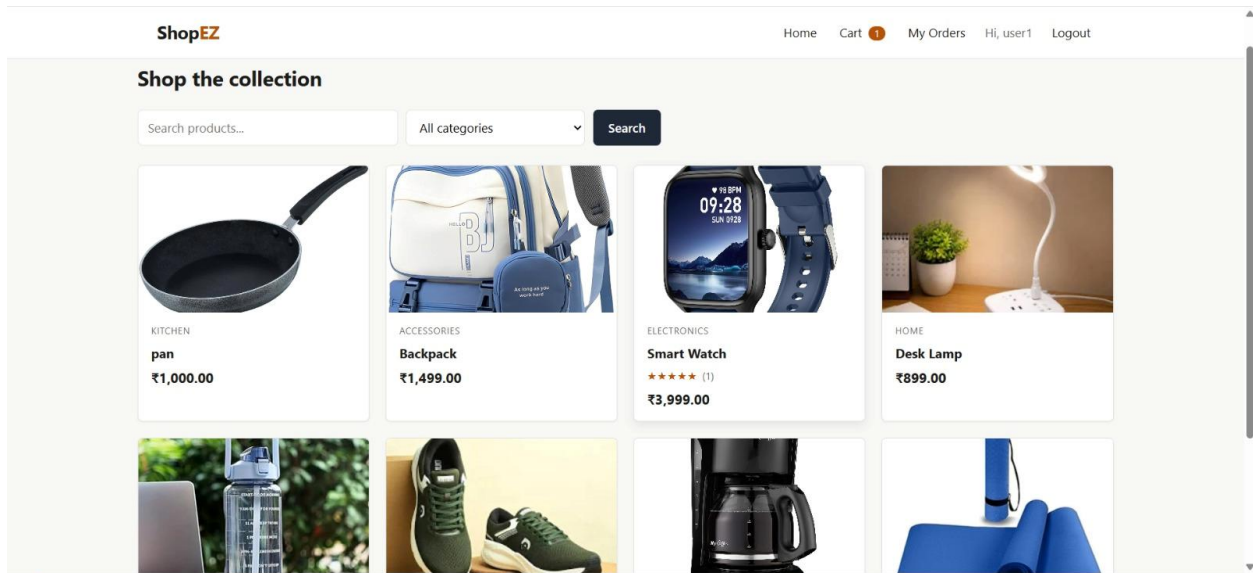
Login

New here? [Create an account](#)

Admin demo login: admin@shopez.com / admin123 (after running the seed script)

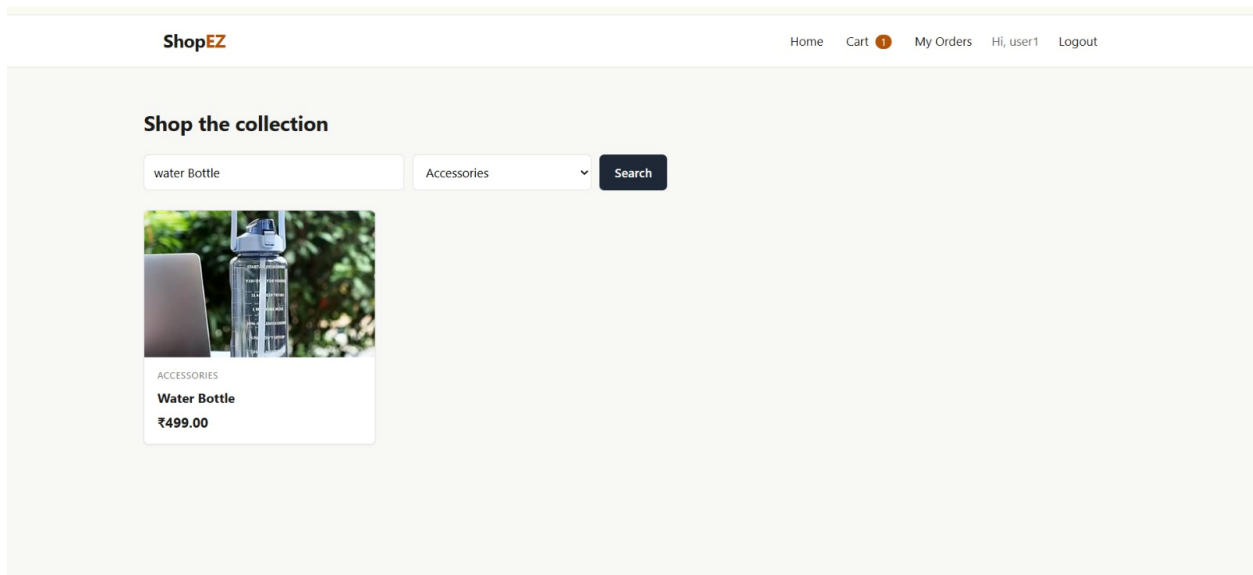
9.3 Home Page / Product Catalog

Full product grid with the category filter dropdown visible.



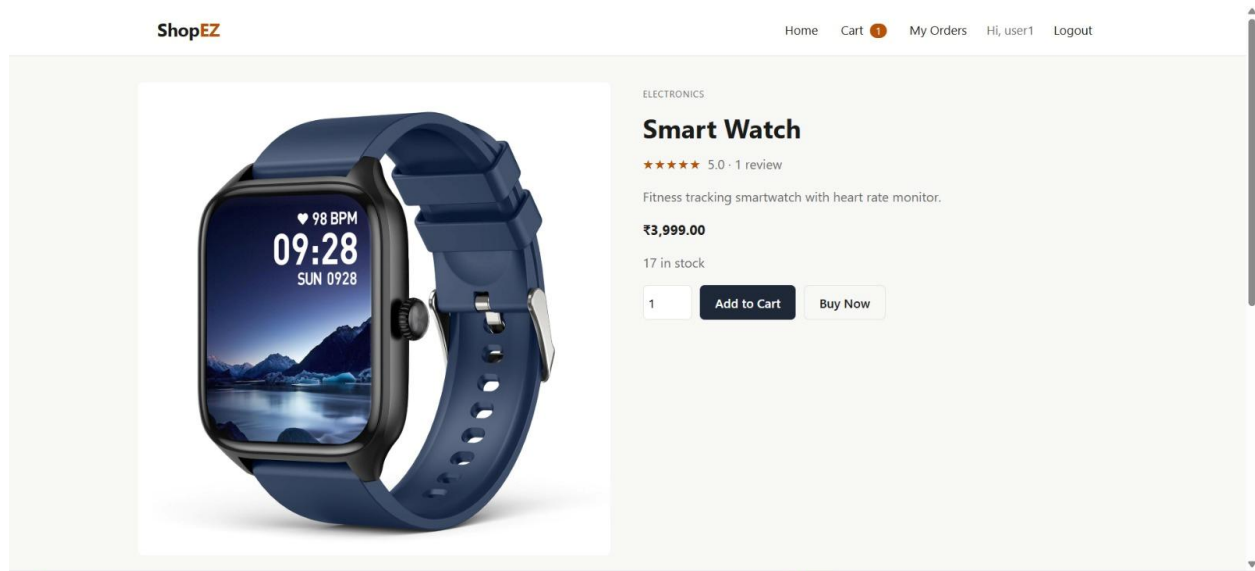
9.4 Product Search & Filter Results

Search box in use with a keyword typed, showing filtered results.



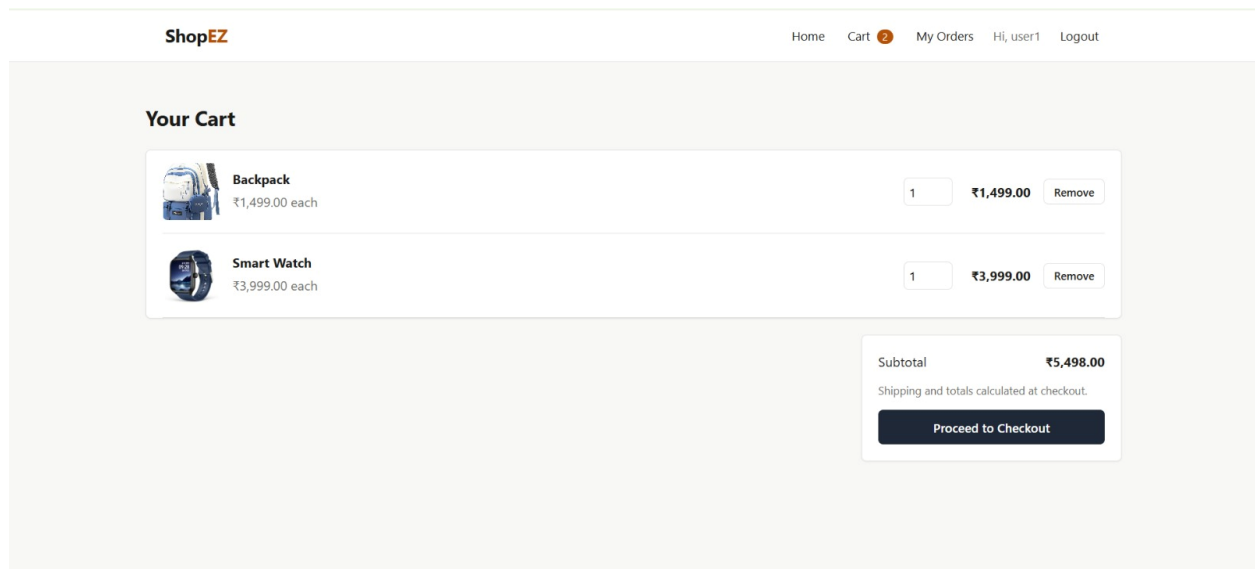
9.5 Product Detail Page

Full product detail view including the reviews & star-rating section at the bottom.



9.6 Shopping Cart Page

Cart with at least 2 items, showing quantity controls and the total price.



9.7 Checkout Page

Shipping address form with the Cash on Delivery order summary.

ShopEZ

Home Cart 1 My Orders Hi, user1 Logout

Checkout

Full Name

Address

City

Postal Code

Country

Place Order (Cash on Delivery)

Order Summary

pan x1	₹1,000.00
Subtotal	₹1,000.00
Shipping	₹0.00
Total	₹1,000.00

9.8 My Orders Page

Order history list showing at least one order and its status.

ShopEZ

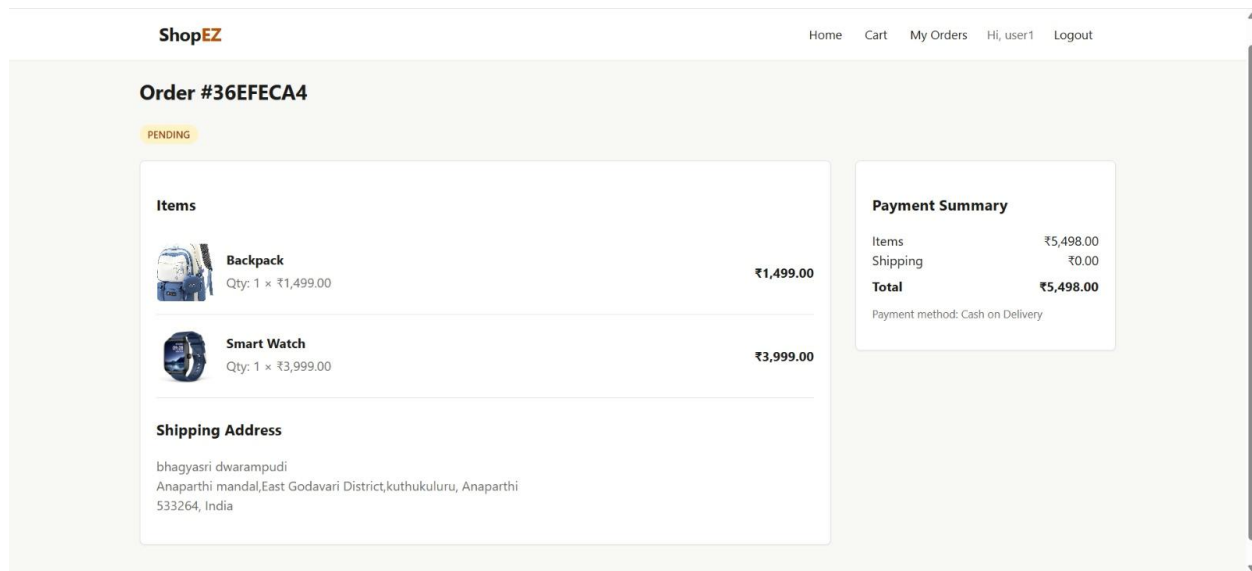
Home Cart 1 My Orders Hi, user1 Logout

My Orders

ORDER ID	DATE	TOTAL	STATUS	
36EFECA4	13/07/2026	₹5,498.00	PENDING	View
648EBE72	05/07/2026	₹3,999.00	PROCESSING	View

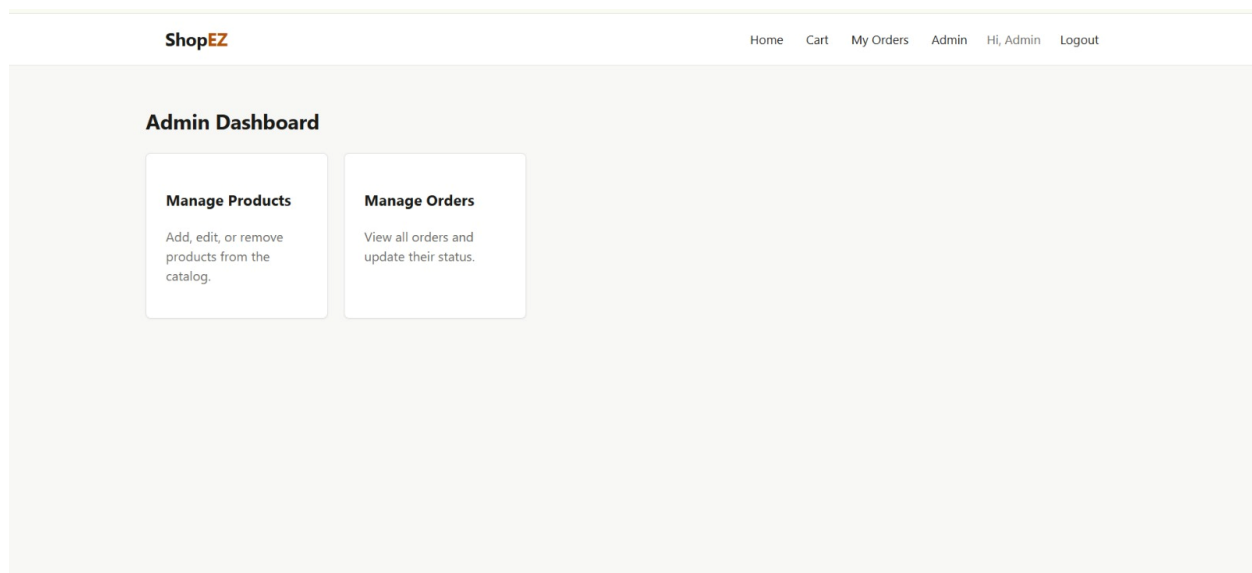
9.9 Order Detail Page

Single order's full detail view, including items and shipping address.



9.10 Admin Dashboard

Admin landing page/overview after logging in as admin@shopez.com.



9.11 Admin – Manage Products

Product list with CRUD controls, and the image upload form open.

ShopEZ

[Home](#)
[Cart 1](#)
[My Orders](#)
[Admin](#)
[Hi, Admin](#)
[Logout](#)

Manage Products

Add New Product

Name

Category

Price (₹)

Stock

Product Image

Choose file No file chosen

Description

Add Product

Add Product

NAME	CATEGORY	PRICE	STOCK		
pan	Kitchen	₹1,000.00	4	Edit	Delete
Backpack	Accessories	₹1,499.00	3	Edit	Delete
Smart Watch	Electronics	₹3,999.00	16	Edit	Delete
Desk Lamp	Home	₹899.00	50	Edit	Delete
Water Bottle	Accessories	₹499.00	100	Edit	Delete
Running Shoes	Footwear	₹1,899.00	40	Edit	Delete
Coffee Maker	Home	₹2,299.00	15	Edit	Delete
Yoga Mat	Fitness	₹699.00	60	Edit	Delete

9.12 Admin – Manage Orders

Orders list with the status-update dropdown/button visible.

ShopEZ
Home
Cart 1
My Orders
Admin
Hi, Admin
Logout

Manage Orders

ORDER ID	CUSTOMER	DATE	TOTAL	STATUS	UPDATE
36FECA4	user1 user@demo.com	13/07/2026	₹5,498.00	PENDING	PENDING
323EDA3C	Alice alice@example.com	13/07/2026	₹3,999.00	SHIPPED	SHIPPED
648EBE72	user1 user@demo.com	05/07/2026	₹3,999.00	PROCESSING	PROCESSING PENDING PROCESSING SHIPPED DELIVERED CANCELLED

10. Testing

Manual functional testing was performed across the core user flows: registration and login, product browsing/search/filtering, add-to-cart and quantity updates, checkout and order creation, order status updates from the admin dashboard, product image upload, and the review/rating submission flow. Backend module loading and the production frontend build were verified after each change. Automated testing (unit tests for controllers, integration tests for API routes, and component tests for the frontend) is not yet implemented — see Future Enhancements.

11. Known Issues

- Checkout only supports a Cash on Delivery flow; no online payment gateway is integrated.
- Uploaded product images are stored on the server's local disk (server/uploads/), which is not persistent across most cloud redeployments — a cloud storage service would be needed for production use.
- Authentication uses a single long-lived access token rather than a short-lived access token with refresh rotation, which is a simpler but less security-hardened approach.
- The frontend does not yet expose pagination controls in the UI, although the backend's product listing endpoint already supports page and limit parameters.

12. Future Enhancements

- Integrate a real online payment gateway (e.g. Razorpay/UPI for the Indian market) alongside Cash on Delivery.
- Move product image storage to a cloud storage service (e.g. Cloudinary or AWS S3).
- Add short-lived access tokens with refresh token rotation for stronger session security.
- Add a wishlist/favorites feature.
- Add automated testing (unit, integration, and component tests).
- Add pagination controls to the product listing UI.